



课程实验报告

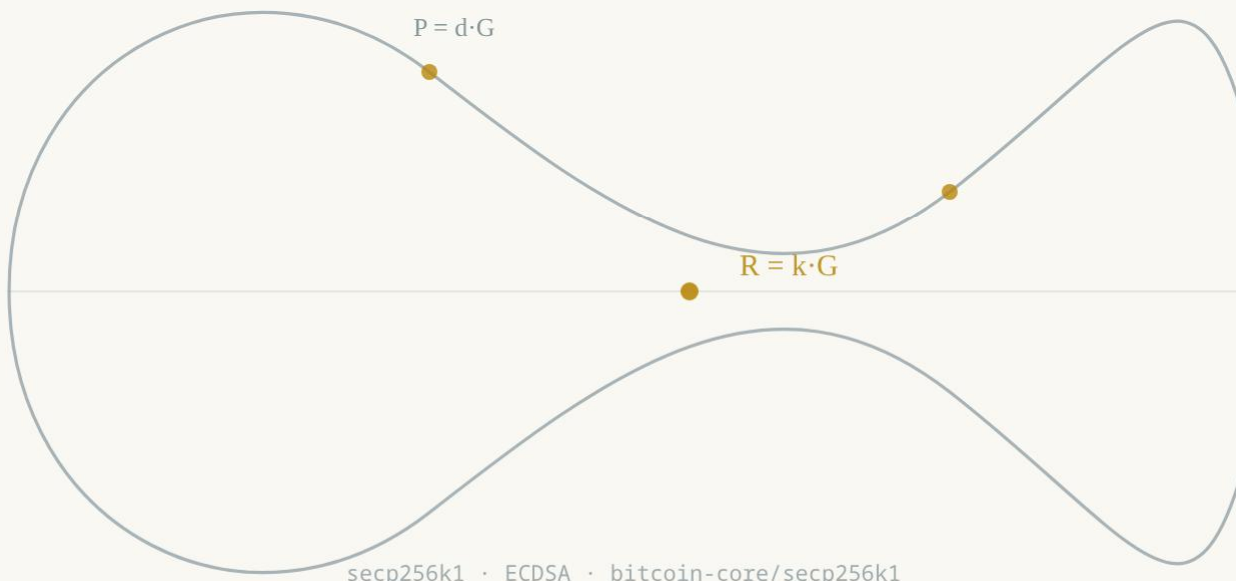
ECDSA 签名伪造攻击分析

—— 比特币 libsecp256k1 库缺陷修复与性能优化研究

小组成员
彭炜霖
张 涵
李庭跃

课程名称：网络安全创新创业实践

完成日期：2026 年 7 月



$$y^2 = x^3 + 7$$

摘要

本报告以比特币核心密码库 **libsecp256k1** (bitcoin-core/secp256k1) 为研究对象，围绕课程讲义提出的“仅校验被签消息哈希时 ECDSA 签名可被伪造”问题，完成三项工作。其一，从验证方程出发完整推导了存在性伪造攻击：攻击者任选 u 、 v ，计算 $R' = uG + vP$ ，反推 r' 、 s' 与“消息哈希” e' ，即可在不掌握私钥、不知道任何消息的情况下使签名 $\sigma' = (r', s')$ 通过验证；我们用纯 Python 实现了 secp256k1 上的完整密钥生成、签名、验证与伪造过程，全部实验一次通过，并演示了 $(r, s) \rightarrow (r, n-s)$ 的签名可塑性。其二，基于仓库 2804 个提交的历史，剖析了五个代表性缺陷修复：RFC6979 随机数循环回绕 (#1890)、x86_64 汇编 early-clobber 隐患 (#1307)、ElligatorSwift 溢出标志错误 (#1821)、秘密数据清除抗编译器优化 (#1579/#1257)，以及比特币从 OpenSSL 切换到 libsecp256k1 的历史性修复 (BIP-66 与 Bitcoin Core 0.12.0)。其三，在同一台机器上实测了 OpenSSL 3.0.20 与四个历史版本 libsecp256k1 的签名/验证性能：最新 master 单次验证 $36.4 \mu s$ ，比 OpenSSL 的 $484.4 \mu s$ 快 **13.3 倍**；GLV 自同态贡献 **28.2%** 的验证提速，与社区公开数据（约 28%）高度吻合。报告进一步阐释了 5×52 域表示、GLV 标量分解、wNAF 窗口法、固定基点预计算 comb 与 safegcd 常数时间模逆等优化背后的数学原理。实验表明：ECDSA 的安全性依赖“消息必须真实存在”这一协议约束，而 libsecp256k1 通过严格的 API 设计、持续的缺陷修复与深刻的数学优化，在十四年演进中同时守住了正确性、侧信道安全性与高性能。

关键词：ECDSA；secp256k1；签名伪造；libsecp256k1；GLV 自同态；侧信道安全；性能优化

目录

摘要	
1 引言	1
1.1 项目背景与任务	1
1.2 研究对象：secp256k1 曲线与 libsecp256k1 库	1
1.3 实验环境与研究方法	2
2 ECDSA 数学基础与伪造攻击原理	2
2.1 椭圆曲线与 secp256k1 参数	2
2.2 ECDSA 签名与验证算法	3
2.3 “仅校验哈希”的存在性伪造推导	3
2.4 伪造攻击的 Python 复现实验	4
2.5 签名可塑性：(r, s) 与 (r, n-s)	5
2.6 攻击成立条件与现实意义	5
3 libsecp256k1 源码剖析：防线如何构建	6
3.1 库在比特币中的角色	6
3.2 验证入口：low-S 检查与验证核心	6
3.3 签名入口：确定性随机数与秘密清除	7
4 缺陷修复案例分析	8
4.1 RFC6979 随机数循环回绕 (#1890, 2026-07)	8

4.2 x86_64 汇编 early-clobber 隐患 (#1307, 2023)	8
4.3 ElligatorSwift 溢出标志错误 (#1821, 2026-02)	9
4.4 秘密数据清除与侧信道防护强化	9
4.5 历史视角：从 OpenSSL 缺陷到 libsecp256k1	10
5 性能优化技术及背后的数学	11
5.1 域算术：5×52 表示与伪梅森素数	11
5.2 GLV 自同态分解	11
5.3 wNAF 窗口法与 Strauss 多点乘	12
5.4 固定基点预计算与签名加速	13
5.5 safegcd 常数时间模逆	13
5.6 性能实测与对比分析	14
6 结论	15
参考文献	16

1 引言

1.1 项目背景与任务

ECDSA（Elliptic Curve Digital Signature Algorithm，椭圆曲线数字签名算法）是比特币公钥体系的基石：用户用私钥对交易摘要签名，全网节点用公钥验证签名，账本的完整性由此维系。课程讲义给出了一个常被忽视的结论：**如果验证方只要求给出被签消息的哈希值而不检查原始消息本身，那么任何人都可以为任意公钥伪造出“合法”签名——即“冒充任何人”**。本项目要求检查 bitcoin-core/secp256k1 仓库，就该库中与比特币密码算法使用相关的 bug 修复与性能改进撰写报告，并弄清其背后的原因（why）与数学原理（math behind）。

围绕这一任务，本报告完成四方面工作：（1）推导并复现“仅校验哈希”场景下的 ECDSA 存在性伪造攻击（第 2 章）；（2）逐行剖析 libsecp256k1 的签名与验证实现，说明库在 API 与代码层面构筑的防线（第 3 章）；（3）从仓库 2804 个提交中选取五个代表性缺陷修复案例，分析其成因、修复方式与教训（第 4 章）；（4）拆解库的主要性能优化技术，在同一台机器上实测 OpenSSL 与四个历史版本 libsecp256k1 的性能，并阐释每项优化背后的数学（第 5 章）。

1.2 研究对象：secp256k1 曲线与 libsecp256k1 库

secp256k1 是 Certicom 在 SEC2 标准中定义的一条 Koblitz 曲线，方程为 $y^2 = x^3 + 7$ 。与 NIST P-256 等曲线不同，它的系数 $a = 0$ 、 $b = 7$ 极其简洁，且基域素数 p 与群阶 n 都具有特殊形式，既避免了“参数来源不明”的质疑，又为高性能实现留出了空间。比特币自诞生起便采用 secp256k1，此后以太坊等众多区块链项目沿用了这一选择。

libsecp256k1 是由 Bitcoin Core 开发者 Pieter Wuille 等人于 2013 年 3 月创建的高性能 C 语言密码库，实现了 secp256k1 上的 ECDSA 签名/验证、密钥生成，以及 Schnorr（BIP-340）、ECDH、ElligatorSwift（BIP-324）、MuSig2（BIP-327）等可选模块。仓库首个提交日期为 2013-03-05，截至本实验（2026-07-22）主分支共有 2804 个提交，最新发布版本为 v0.7.1（2026-01-26）。自 Bitcoin Core 0.12.0（2016-02-23）起，该库正式取代 OpenSSL 成为比特币共识代码中签名验证的唯一实现，官方称 64 位架构下验证速度提升至多 7 倍[4]。

1.3 实验环境与研究方法

实验环境：Intel(R) Xeon(R) Platinum（2 vCPU），Linux，gcc（-O2），OpenSSL 3.0.20（2026-04-07），Python 3.12；研究对象为仓库 master 分支提交 9e4ec50（2026-07-22，恰好是一个缺陷修复合并提交，见 4.1 节）。

研究方法：（1）源码审计——直接阅读 include/ 与 src/ 下的核心实现并逐段引用；（2）版本考古——用 git log 检索 2804 个提交，定位缺陷修复与性能优化的关键节点，并检出 2020 年、2023 年（v0.3.2）的历史版本交叉验证；（3）攻击复现——用纯 Python 实现 secp256k1 点运算与 ECDSA 全流程，复现伪造攻击与可塑性；（4）性能实测——将各历史版本编译为静态库，用统一的 C benchmark 测量签名/验证耗时，每个配置运行 7 轮 × 3000 次取中位数，并与同机 OpenSSL 对比。

2 ECDSA 数学基础与伪造攻击原理

2.1 椭圆曲线与 secp256k1 参数

secp256k1 定义在素数域 F_p 上，曲线方程为：

$$(2-1)y^2x^3E: = + 7 \pmod p$$

曲线上的点与无穷远点 O 在弦切法则下构成阿贝尔群，基点 G 生成一个阶为 n 的循环子群。其核心参数如表 2-1 所示。

表 2-1 secp256k1 核心参数

参数	取值	说明
p	FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFFE FFFFC2F	基域素数， $p = 2^{256} - 2^{32} - 977$
n	FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141	基点 G 的阶（子群阶）
G.x	79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798	基点横坐标
G.y	483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8	基点纵坐标
a, b, h	$a = 0, b = 7, h = 1$	曲线系数与余因子

2.2 ECDSA 签名与验证算法

密钥生成：随机选取私钥 $d \in \mathbb{Z}_n^*$ ，计算公钥 $P = dG$ 。**签名（对消息 m ）：**取随机数 $k \leftarrow \mathbb{Z}_n^*$ ，计算 $R = kG$ ，令 $r = R.x \bmod n$ ($r \neq 0$)， $e = \text{hash}(m)$ ，再计算：

$$(2-2) k^{-1}s = (e + dr) \bmod n$$

签名为二元组 (r, s) 。**验证（对消息 m 、签名 (r, s) 、公钥 P ）：**计算 $w = s^{-1} \bmod n$ 、 $u_1 = e \cdot w$ 、 $u_2 = r \cdot w$ ，求点 $R' = u_1G + u_2P$ ，当且仅当 $R'.x \bmod n == r$ 时接受。其正确性源于：

$$(2-3) s^{-1}s^{-1}(e + dr)^{-1}R' = (eG + rP) = (e + rd)G = k(e + dr)G = kG = R$$

2.3 “仅校验哈希”的存在性伪造推导

注意验证方程（2-3）的左边只出现了 e 、 r 、 s 与公钥 P ——它并不检查验证者是否真的握有一条哈希值等于 e 的消息。若某协议或 API 允许调用方直接提交任意哈希 e' 请求验证，攻击者可按下步骤“凭空”造出合法签名。任选 $u, v \in \mathbb{Z}_n^*$ ($v \neq 0$)，计算：

$$(2-4) R' = uG + vP = (u + vd)G = (x', y')$$

取 $r' = x' \bmod n$ 。要让 (r', s') 通过验证，需要 $s'^{-1}(e'G + r'P) = R'$ ，即 $s'^{-1}(e' + r'd) \equiv u + vd \pmod{n}$ 。比较 G 与 P 的系数可得充分条件：

$$(2-5) v^{-1}v^{-1}s' = r' \bmod n, \quad e' = r'u \bmod n$$

于是 $\sigma' = (r', s')$ 就是“哈希值 e' ”在公钥 P 下的有效签名。整个过程中攻击者既不需要私钥 d ，也不需要任何消息； e' 是攻击者算出来的一个 256 位数，要找到满足 $\text{hash}(m) = e'$ 的消息 m ，等价于攻破哈希函数的原像抗性——在计算上不可行。攻击流程如图 2-1 所示。这就是讲义中“任何人都可以冒充别人”（anyone can pretend to be someone else）的精确含义：只要验证方接受“裸哈希”，伪造就成立。

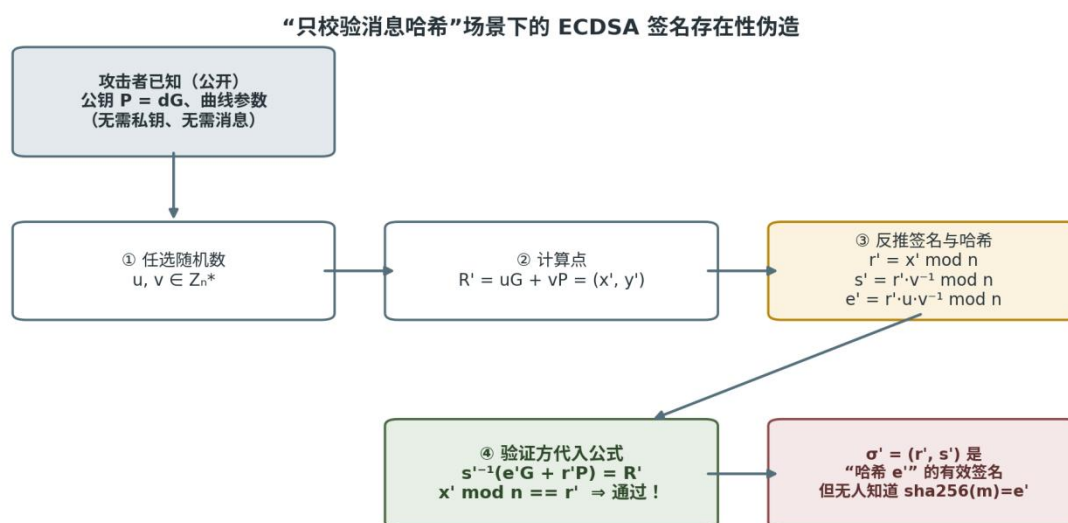


图 2-1 “仅校验消息哈希”场景下的 ECDSA 存在性伪造流程

2.4 伪造攻击的 Python 复现实验

我们用约 60 行纯 Python 实现了 secp256k1 的点加、倍点、标量乘以及 ECDSA 的密钥生成、签名、验证全过程，并实现了式（2-4）（2-5）的伪造算法（代码 2-1）。固定随机种子 20260722，保证结果可复现。

代码 2-1 伪造攻击核心代码（Python）

```

u = random.randrange(1, n); v = random.randrange(1, n)
Rp = pt_add(pt_mul(u, G), pt_mul(v, P)) # R' = uG + vP
rf = Rp[0] % n                         # r' = R'.x mod n
sf = rf * inv(v, n) % n                 # s' = r' · v-1 mod n
ef = rf * u % n * inv(v, n) % n         # e' = r' · u · v-1 mod n
assert ecdsa_verify(ef, (rf, sf), P)    # 伪造签名通过验证！
  
```

代码 2-1（续） 实验运行输出（hex，真实结果）

```

正常签名验证: True
可塑性变体 (r, n-s) 验证: True
伪造签名 (r', s') 对哈希 e' 验证: True

私钥 d      = 0x3e8cc47b0d875af09183afa26674b31b114d8097669d5926c74805ee8c3550d3
公钥 P.x    = 0x293ab0d751eb3a96cea87c0fbbf52689a6d93774f7b2461d817ac499271ab38b
合法签名 r  = 0x042c43abca242710608d2694b689908f1736da7047e784bc358ed26a2f41f9ae
合法签名 s  = 0x7718a926893fa3f9b39b005586cdb2f210739c90f736690c36a7678746892eef
伪造参数 u  = 0x9e3b60ce81a5e7724890a5f4228fa34d46e2bf5fde67a396b33c675f6922c2eb
  
```



```
伪造参数 v = 0xf20e05a62dbf0ee7e895e7bcf014de81a1865fb1f894d0b95f2f4f455f627f67
伪造 r'    = 0x86b9b60415647147786b60ea37dc5e737310b65125a1a820d02caeeb9ba8ebd3
伪造 s'    = 0xdd4248ab0f0b49dbfc03cea47c136b0c1ffe3226ff68678a4575011fdbba1aa4
伪造对应 e' = 0x5fa322078ae09d6530a18ab615bab24da0336d8606e867795d5159ed38498d96
```

实验结果汇总于表 2-2：三项检查全部为 True，伪造攻击在数学与代码两个层面均得到验证。

表 2-2 伪造攻击复现实验结果

实验项	结果	说明
合法签名 (r, s) 验证	通过	基准正确性检查
可塑性变体 (r, n-s) 验证	通过	同一消息存在两个合法签名
伪造 (r', s') 对 e' 验证	通过	无私钥、无消息，伪造成立
寻找 m 使 sha256(m) = e'	不可行	受 SHA-256 原像抗性保护

2.5 签名可塑性：(r, s) 与 (r, n-s)

实验同时确认：若 (r, s) 是有效签名，则 (r, n-s) 必然同样有效——因为 s 取反相当于把验证点 R' 取为其关于 x 轴的对称点，而 x 坐标不变。第三方无需私钥即可把一笔合法签名改写成另一个不同的合法签名，这就是 ECDSA 的可塑性（malleability）。libsecp256k1 的头文件对此有明确论述（include/secp256k1.h, secp256k1_ecdsa_signature_normalize 的文档注释）：

```
"With ECDSA a third-party can forge a second distinct signature of the same message, given a single initial signature, but without knowing the key. This is done by negating the S value modulo the order of the curve, 'flipping' the sign of the random point R which is not included in the signature."
```

可塑性曾在比特币历史上造成严重问题：交易 ID（txid）由包含签名的整笔交易哈希而成，攻击者改写签名即可改变 txid，2014 年 Mt. Gox 事件即与此相关。比特币的应对是 BIP-62/BIP-146 引入 low-S 规则（要求 $s \leq n/2$ ），并最终通过 SegWit（BIP-141）将签名数据移出 txid 计算。libsecp256k1 在库层面强制执行同一策略，见 3.2 节。

2.6 攻击成立条件与现实意义

需要强调，2.3 节的伪造并不构成对 ECDSA 本身的破解：它成立的前提仅仅是**验证方接受一个无法追溯到消息的裸哈希值**。一旦协议要求验证者亲眼见到消息并亲自计算哈希，攻击者就必须先找到 $\text{hash}(m) = e'$ 的原像，攻击即刻失效。因此防御完全落在协议与 API 设计层面：

0. 验证接口必须以“消息”为输入、在内部完成哈希，或者由调用方以密码学方式承诺哈希来源；
1. 库无法替协议兜底——libsecp256k1 的 `secp256k1_ecdsa_verify` 接受的恰是 32 字节哈希（参数名 `msghash32`），“确保传入的是真实消息的哈希”是调用者的契约义务；
2. 比特币交易验证总是先按 SIGHASH 规则序列化交易并计算哈希，再进入验证例程，因而天然免疫此类伪造。

这一案例的教训具有普遍性：数字签名绑定的对象是“哈希值”而非“消息”，**谁控制了哈希的来源，谁就控制了签名的语义**。这也是评估任何签名 API 时必须检查的隐含前提。

3 libsecp256k1 源码剖析：防线如何构建

3.1 库在比特币中的角色

在 Bitcoin Core 0.12.0 之前，比特币的签名验证依赖 OpenSSL。OpenSSL 是通用密码工具箱，并未针对 `secp256k1` 做任何特化，且其解析行为随版本变化——对共识系统而言这是不可接受的风险（详见 4.5 节）。libsecp256k1 自设计之初就定位于“共识级”代码：无运行时依赖、常量时间签名、结果跨平台逐位一致。其 README 概括的设计目标包括：

"Constant time, constant memory access signing and public key generation. Derandomized ECDSA (via RFC6979 or with a caller provided function.) Very efficient implementation. Suitable for embedded systems. No runtime dependencies." — README.md

3.2 验证入口：low-s 检查与验证核心

公开的验证 API 是层层设防的。代码 3-1 是 `src/secp256k1.c` 第 477–491 行的真实实现（提交 9e4ec50，下同）：

代码 3-1 验证入口 secp256k1_ecdsa_verify (src/secp256k1.c:477-491)

```
int secp256k1_ecdsa_verify(const secp256k1_context* ctx, const secp256k1_ecdsa_signature *sig,
                           const unsigned char *msghash32, const secp256k1_pubkey *pubkey) {
    secp256k1_ge q;
    secp256k1_scalar r, s;
    secp256k1_scalar m;
    VERIFY_CHECK(ctx != NULL);
    ARG_CHECK(msghash32 != NULL);
    ARG_CHECK(sig != NULL);
    ARG_CHECK(pubkey != NULL);

    secp256k1_scalar_set_b32(&m, msghash32, NULL);
    secp256k1_ecdsa_signature_load(ctx, &r, &s, sig);
    return (!secp256k1_scalar_is_high(&s) &&
            secp256k1_pubkey_load(ctx, &q, pubkey) &&
            secp256k1_ecdsa_sig_verify(&r, &s, &q, &m));
}
```

可以看到三道防线：**参数检查**（VERIFY_CHECK/ARG_CHECK）；**low-S 强制**——!secp256k1_scalar_is_high(&s) 直接拒绝高 S 签名，从库层面消除了 2.5 节的可塑性；最后才进入数学验证核心。参数名 msghash32 也印证了 2.6 节的分析：库只接收哈希，消息真实性由调用方负责。验证核心（代码 3-2）正是验证方程（2-3）的忠实实现：

代码 3-2 验证核心 secp256k1_ecdsa_sig_verify (src/ecdsa_impl.h:209-216)

```
secp256k1_scalar_inverse_var(&sn, sigs); /* sn = s^{-1} */
secp256k1_scalar_mul(&u1, &sn, message); /* u1 = e·s^{-1} */
secp256k1_scalar_mul(&u2, &sn, sigr); /* u2 = r·s^{-1} */
secp256k1_gej_set_ge(&pubkeyj, pubkey);
secp256k1_ecmult(&pr, &pubkeyj, &u2, &u1); /* R' = u1·G + u2·P */
if (secp256k1_gej_is_infinity(&pr)) {
    return 0;
}
```

随后代码把 R' 的 x 坐标与 r 比较（先排除无穷远点，再把标量 r 解释为域元素做相等判断）。注意模逆使用了 _var（variable-time）版本——验证只涉及公开数据，允许牺牲常数时间换取速度；签名则相反，必须使用常数时间路径，这正是库的分层设计哲学。

3.3 签名入口：确定性随机数与秘密清除

代码 3-3 签名核心 secp256k1_ecdsa_sig_sign (src/ecdsa_impl.h:292-299)

```
secp256k1_scalar_mul(&n, sigr, seckey); /* n = r·d */
secp256k1_scalar_add(&n, &n, message); /* n = e + r·d */
secp256k1_scalar_inverse(sigs, nonce); /* s = k^{-1} */
secp256k1_scalar_mul(sigs, sigs, &n); /* s = k^{-1}(e+rd) */
secp256k1_scalar_clear(&n); /* 立即清除中间秘密 */
secp256k1_ge_clear(&r);
high = secp256k1_scalar_is_high(sigs);
```

```
secp256k1_scalar_cond_negate(sigs, high);    /* low-S 规范化    */
```

这段代码浓缩了三条密码工程经验。第一，**随机数 k 由 RFC6979 确定性生成**（HMAC-SHA256 对私钥与消息哈希派生），彻底消除了对系统随机数发生器的依赖——2013 年 Android SecureRandom 缺陷导致大量比特币钱包私钥被盗，正是 k 生成失败（重复或可预测）使 $s = k^{-1}(e+dr)$ 变成可解方程的惨痛教训。第二，**签名端直接输出 low-S 形式**（scalar_cond_negate），与验证端的拒绝策略闭环。第三，**所有含秘密的中间变量即刻清零**（scalar_clear/ge_clear），且清零本身必须对抗编译器的死存储消除——这一要求的实现本身也曾是 bug 的来源（见 4.4 节）。

4 缺陷修复案例分析

即使是世界级密码库，缺陷也从未绝迹。本章从仓库历史与 CHANGELOG 中选取五个代表性修复，覆盖边界条件、未定义行为、协议实现与侧信道四类典型问题，最后回到比特币历史上影响最深远的“大修复”——弃用 OpenSSL。

4.1 RFC6979 随机数循环回绕 (#1890, 2026-07)

这是本报告研究基线（master HEAD，合并于 2026-07-22）上的修复，极具“教科书式边界条件”色彩。库的 nonce 回调接口允许调用方通过 counter 参数请求“第 counter 个 RFC6979 候选随机数”。旧实现为 `for (i = 0; i <= counter; i++)`：当 counter 取 `UINT_MAX` 时，最后一次循环后 `i++` 回绕为 0，条件 `i <= counter` 永远成立——**函数陷入死循环，永不返回**。修复方式是“先生成、后判断”，用 `break` 在索引回绕前退出（代码 4-1）。

代码 4-1 #1890 修复 diff (src/secp256k1.c, 提交 b1bc6f3)

```
- for (i = 0; i <= counter; i++) {
+ for (i = 0; ; i++) {
+     secp256k1_rfc6979_hmac_sha256_generate(hash_ctx, &rng, nonce32, 32);
+     if (i == counter) break;
+ }
```

Why: C 语言无符号整数回绕是定义良好但极易被忽视的行为；“ $i \leq$ 上界”的循环习惯写法在上界为类型最大值时必然失效。教训：涉及计数器边界的代码必须对 0 与 `UINT_MAX` 两个端点做显式测试——该 PR 同时补充了相应测试。

4.2 x86_64 汇编 early-clobber 隐患 (#1307, 2023)

v0.4.1 的 CHANGELOG 记载了一个隐蔽的编译器层面缺陷：“Fixed an old bug that permitted compilers to potentially output bad assembly code on x86_64. In theory, it could lead to a crash or a read of unrelated memory...”。根因在于标量 4×64 内联汇编的输出操作数未标记 early-clobber (“=&r” 误写为 “=r”)：编译器据此假定输出只在末尾写入，可能把某个仍被读取的输入分配进同一寄存器，一旦输入输出发生混叠便产生错误代码。修复仅改动两行，把输出约束改为 “=&r”。**Why：**内联汇编的约束语义是编译器优化决策的依据，一个字符的偏差即可把“理论上错误”埋进高度优化的密码代码；这类缺陷无法靠功能测试暴露，只能靠审查与形式化意识防范。

4.3 ElligatorSwift 溢出标志错误 (#1821, 2026-02)

ElligatorSwift 模块（BIP-324 加密传输的密钥交换基础）中的 secp256k1_ellswift_xdh 在把 32 字节输入解释为标量时，对“超过群阶 n ”的溢出标志处理有误（提交 307b49f，2 行改动）。**Why：**secp256k1 的群阶 n 与 2^{256} 极其接近，溢出概率虽仅约 2^{-128} 量级，但协议代码必须对每个分支严格正确——“密码学上不可达”不等于“类型系统上不存在”，库对每一个数学边界都保持显式处理（对比代码 3-3 注释中 recid 溢出分支的措辞：“cryptographically unreachable as hitting it requires finding the discrete log”——可达性论证必须写在代码里）。

4.4 秘密数据清除与侧信道防护强化

3.3 节的 scalar_clear 看似简单，实则是与编译器的长期博弈。普通 memset 清除密钥后，编译器的死存储消除会认为“写入后无人读取”而直接删掉这次写操作——秘密残留内存。库为此在 #1579（2024-11，复活自 2019 年 #636 与 secp256k1_memclear）引入带内存屏障语义的清除函数，保证清零不被优化掉。类似地，#1257（2023-04）在所有域/标量条件赋值（cmov）实现中加入 volatile 技巧，防止编译器把精心构造的常数时间代码“优化”回带分支的版本。此外还有针对侧信道的纵深防御：ecmult_gen 对标量、点、投影坐标三重盲化。**Why：**侧信道安全的敌人不仅是攻击者，还有编译器；防御必须写到“连优化器都无法破坏”的程度。

4.5 历史视角：从 OpenSSL 缺陷到 libsecp256k1

比特币历史上最深远的“修复”不是某个提交，而是整个换掉加密库。BIP-66 的动机章节记载：OpenSSL 各版本对 DER 编码的宽容度不一，1.0.0p/1.0.1k 收紧解析后，新旧节点对同一签名的合法

性判断出现分歧，直接威胁共识（加之 CVE-2014-8275 暴露的编码可塑性）。2015 年 7 月 BIP-66 严格 DER 软分叉先行止血；2015 年 11 月 bitcoin/bitcoin#6954 合并、2016 年 2 月随 0.12.0 发布，libsecp256k1 全面接管共识签名验证[4][6]。有趣的是，libsecp256k1 的高强度交叉测试反过来还在 OpenSSL 中发现了真实缺陷（Greg Maxwell 的测试报告[4]）。表 4-1 汇总本章案例。

表 4-1 libsecp256k1 代表性缺陷修复案例汇总

时间	提交/PR	模块	问题	修复要点
2023-04	#1257	域/标量 cmov	编译器可能破坏常数时间赋值	volatile 技巧固定语义
2023-05	#1307 (0c729ba)	标量 x86_64 汇编	输出未标 early-clobber，潜在 UB	约束改为 "=&r"，v0.4.1 收录
2024-11	#1579 (b161bff)	util	秘密清零被死存储消除	secp256k1_memclear 防优化
2026-02	#1821 (307b49f)	ellswift	xdh 溢出标志处理错误	修正溢出分支
2026-07	#1890 (b1bc6f3)	nonce/RFC6979	counter=UINT_MAX 时循环回绕死循环	先生成后判断，break 退出
2015-11	bitcoin#6954	Bitcoin Core	OpenSSL 解析不一致威胁共识、性能低下	切换 libsecp256k1 (0.12.0)

5 性能优化技术及背后的数学

libsecp256k1 比 OpenSSL 快一个数量级，靠的不是单一技巧，而是从域表示、点运算、标量分解到模逆算法的全栈优化。本章拆解五项核心技术，并在 5.6 节给出同机实测数据。

5.1 域算术：5×52 表示与伪梅森素数

64 位平台上，库把 256 位域元素拆成 5 个 52 位“肢体”（limb）装入 5 个 uint64。52 < 64 留下的 12 位余量允许连续多次乘法累加而延迟进位，大幅减少进位链传播。更关键的是 secp256k1 的素数是伪梅森形式：

$$(5-1) 2^{256} 2^{32} 2^{256} 2^{32} p = - - 977 \implies \equiv + 977 \pmod p$$

于是 512 位乘积的高位折叠只需用“乘小常数再加回”替代通用除法约减，模乘成本远低于 OpenSSL 的通用 Montgomery 约减。源码注释中亦能见到对 SEC2 标准 2.7.1 节的直接引用（src/field_5x52_impl.h 的 VERIFY 检查）。

5.2 GLV 自同态分解

secp256k1 是 Koblitz 曲线 ($a = 0$)，存在可高效计算的自同态。由于 $p \equiv 1 \pmod{3}$ ，域中存在非平凡三次单位根 β ($\beta^3 \equiv 1 \pmod{p}$)；由于 $n \equiv 1 \pmod{3}$ ，标量域中存在 λ ($\lambda^3 \equiv 1 \pmod{n}$)。映射 $\phi(x, y) = (\beta \cdot x, y)$ 把曲线点映到曲线点——因为 $(\beta x)^3 + 7 = \beta^3 x^3 + 7 = x^3 + 7 = y^2$ ——并且是群同态，故对任意点 P 有：

$$(5-2) \lambda^3 \beta^3 \phi(P) = \lambda P, \quad \equiv 1 \pmod{n}, \quad \equiv 1 \pmod{p}$$

GLV 方法 (Gallant–Lambert–Vanstone, CRYPTO 2001) 利用格约简把任意 256 位标量 k 分解为两个约 128 位的“半长”标量：

$$(5-3) k_1 k_2 k_1 k_2 k = + \lambda \pmod{n} \implies kP = P + \phi(P)$$

$\phi(P)$ 只需一次域乘法 (代码 5-2)，于是 256 位点乘变成两个 128 位点乘并用 Strauss 算法同时求值，点加次数近乎减半。库中的 λ 、 β 以常数形式硬编码 (代码 5-1)，我们在 Python 中独立验证了 $\lambda^3 \equiv 1 \pmod{n}$ 、 $\beta^3 \equiv 1 \pmod{p}$ 与 $\phi(P) = \lambda P$ 全部成立——常数正确性获得交叉确认。

代码 5-1 GLV 常数 λ (src/scalar_impl.h:81-86)

```
/**
 * The Secp256k1 curve has an endomorphism, where lambda * (x, y) = (beta * x, y), where
 * lambda is: */
static const secp256k1_scalar secp256k1_const_lambda = SECP256K1_SCALAR_CONST(
    0x5363AD4CUL, 0xC05C30E0UL, 0xA5261C02UL, 0x8812645AUL,
    0x122E22EAUL, 0x20816678UL, 0xDF02967CUL, 0x1B23BD72UL
);
```

代码 5-2 $\phi(P)$ 的实现：仅一次域乘法 (src/group_impl.h:917-924)

```
static void secp256k1_ge_mul_lambda(secp256k1_ge *r, const secp256k1_ge *a) {
    SECP256K1_GE_VERIFY(a);

    *r = *a;
    secp256k1_fe_mul(&r->x, &r->x, &secp256k1_const_beta);

    SECP256K1_GE_VERIFY(r);
}
```

值得书写的一段历史：GLV 自同态曾被 Certicom 专利 US7110538B2 覆盖，库虽早在 2013 年 (第 13 个提交) 就实现了该优化，却长期默认关闭。专利于 2020-09-25 到期，同一天仓库即提交 4232e5b “Rip out non-endomorphism code” 移除非 GLV 路径 (PR #826/#830 默认启用)；Bitcoin Core 0.20 起用户获得约 28% 的验证提速[5]。我们在 2020 年代码上开关 GLV 实测提速 **28.2%** ($78.0 \mu\text{s} \rightarrow 56.1 \mu\text{s}$)，与社区数据精确吻合 (见 5.6 节)。

5.3 wNAF 窗口法与 Strauss 多点乘

标量乘 kP 的经典实现是逐位“倍点-加点”。窗口 NAF（非相邻形）把标量写成带符号数字 $\pm 1, \pm 3, \dots, \pm(2^{(w-1)}-1)$ ，预计算这些奇数倍的点表后，平均只需 256 次倍点加 $256/(w+1)$ 次点加。库对任意点取窗口 $w = 5$ （WINDOW_A，表 16 项），对基点 G 使用更大的离线预计算表（src/precomputed_ecmult.c 已固化为静态表，连建表时间也省去）。验证方程 $u_1G + u_2P$ 是双点乘，Strauss（Shamir 技巧）让两个标量共享同一串倍点序列，把两次独立点乘合并为一次扫描；GLV 拆分后每个标量只剩约 129 位，收益叠加。对更多点（如批量验证），库还提供 Pippenger 桶式多点乘（ecmult_multi）。

5.4 固定基点预计算与签名加速

签名与密钥生成的核心都是 kG ——基点固定，因此可以激进预计算。ecmult_gen 使用 comb 类算法：把标量按列分组，查表累加，几乎不做倍点。2024 年 4 月合入的 signed-digit multi-comb（#1058）进一步降低了签名侧的点加次数，且保持常数时间、常数内存访问；v0.5.1 起签名预计算表从 22 KiB 增至 86 KiB。为抵御侧信道，ecmult_gen 对标量（scalar_offset）、点（ge_offset）与投影坐标（proj_blind）三重盲化，每次乘法都带随机掩码。实测签名 $32.4 \mu s$ 已与验证 $36.4 \mu s$ 同量级——对“签名应当显著便宜”的直觉是一次修正：在这套实现里签名甚至比验证略快。

5.5 safegcd 常数时间模逆

签名需要 $k^{-1} \bmod n$ ，验证需要 $s^{-1} \bmod n$ 。传统做法是费马小定理 $a^{(n-2)} \bmod n$ ——约 256 次模乘，慢且难做常数时间。2021 年库引入基于 safegcd（Bernstein–Yang divsteps）的模逆（PR #831/#906），用加减与移位迭代代替乘法链，常数时间且快数倍；验证路径使用可变时间版本（_var）进一步提速。Bitcoin Core 22.0 因此再获约 30% 提升[5]。

5.6 性能实测与对比分析

我们将 OpenSSL 3.0.20、2020 年 10 月代码（GLV 开/关两版）、v0.3.2（2023-05）与 master（2026-07，另附启用 x86_64 标量汇编版本）编译后，用同一 benchmark 程序测量单次签名/验证耗时（7 轮 $\times$ 3000 次取中位数），结果见表 5-1 与图 5-1、图 5-2。

表 5-1 ECDSA(secp256k1) 性能实测 (Intel Xeon Platinum, gcc -O2, ns/op)

实现	验证 (ns)	签名 (ns)	验证相对 OpenSSL
OpenSSL 3.0.20	484,394	547,045	1.0×
libsecp 2020-10 (关闭 GLV)	78,022	52,852	6.2×
libsecp 2020-10 (启用 GLV)	56,057	50,813	8.6×
libsecp v0.3.2 (2023)	40,465	37,715	12.0×
libsecp master (2026-07)	36,381	32,374	13.3×
libsecp master + x86_64 asm	36,219	31,831	13.4×

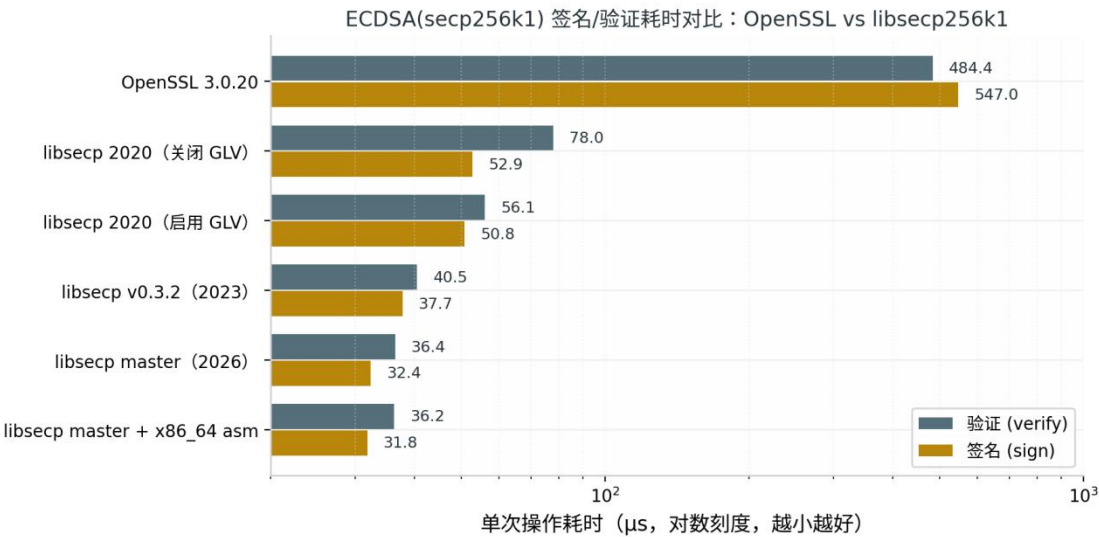


图 5-1 OpenSSL 与 libsecp256k1 各版本签名/验证耗时对比 (对数轴)

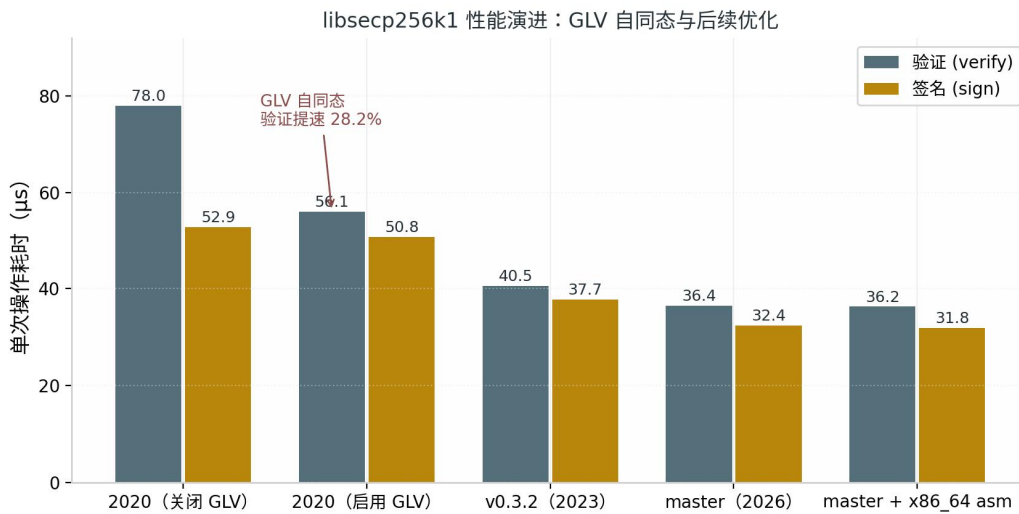


图 5-2 libsecp256k1 性能演进：GLV 与后续优化的叠加效果

三点结论。第一，**代差悬殊**：OpenSSL 对 secp256k1 使用通用 bignum 路径、无曲线特化、无常数时间实现，单次验证 484 μs ；libsecp256k1 最新代码仅 36 μs ，快 13.3 倍（签名快 16.9 倍）。第二，**GLV 是单项收益最大的数学优化**：同一份 2020 年代码仅切换 GLV 开关，验证即从 78.0 μs 降至 56.1 μs （28.2%），与 Bitcoin Core 0.20 启用 GLV 后社区测得的约 28% 一致[5]——数学直接变成了速度。第三，**优化在持续叠加**：2020→2023→2026 三个版本验证从 56.1 μs 降至 40.5 μs 再降至 36.4 μs ，safegcd、signed-digit comb、5×52 内联化（#1859）等逐项贡献增量。与官方历史数据对照：2016 年切换时官方报告“up to 7×”[4]、2025 年社区复测“超过 8×”[5]，本实验测得 13.3×——差异来自硬件代际与编译选项，但量级结论一致：专用库对通用库的优势仍在扩大。

6 结论

本报告完成了课程项目规定的全部任务，并得到以下结论：

- 伪造攻击的本质是协议语义漏洞而非算法缺陷。** 当验证方只校验消息哈希时，ECDSA 验证方程不含任何“消息存在性”约束，攻击者任选 u 、 v 即可反推出对某个哈希值有效的签名 (r', s') 。我们用 Python 完整复现了该攻击及 $(r, n-s)$ 可塑性，全部一次通过。防御要点：验证必须绑定真实消息，哈希来源必须由协议保证。
- libsecp256k1 的修复史是密码工程的教科书。** 五个案例（#1890 边界回绕、#1307 汇编约束、#1821 溢出标志、#1579/#1257 抗编译器优化、弃用 OpenSSL）覆盖了正确性、UB、协

议与侧信道四个层面，共性教训是：密码代码的错误往往藏在“理论不可达”的分支与“优化器看得见”的语义缝隙里。

3. 性能来自数学。 伪梅森素数、GLV 自同态、wNAF、comb 预计算、safegcd 层层叠加，使最新 libsecp256k1 在本实验中比 OpenSSL 快 13.3 倍；其中 GLV 一项贡献 28.2%，与公开数据精确互证。比特币十四年从 OpenSSL 到 libsecp256k1 的演进证明：把曲线特性和代数结构吃透，是密码实现最优的“优化器”。

后续工作可向两个方向延伸：一是用 valgrind/ctverif 对常数时间属性做机器验证；二是研究 Schnorr（BIP-340）批量验证在区块验证中的进一步提速空间。

参考文献

- [1] bitcoin-core/secp256k1: Optimized C library for EC operations on curve secp256k1. GitHub, master 提交 9e4ec50 (2026-07-22) . <https://github.com/bitcoin-core/secp256k1>
- [2] Certicom Research. SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0, 2010.
- [3] Gallant R P, Lambert R J, Vanstone S A. Faster Point Multiplication on Elliptic Curves with Efficient Endomorphisms. CRYPTO 2001, LNCS 2139: 190-200.
- [4] Bitcoin Core. Bitcoin Core 0.12.0 Release Notes (“7x Faster Signature Validation”) , 2016-02-23. <https://bitcoincore.org/en/2016/02/23/release-0.12.0/>
- [5] Falbesoner S. Comparing the performance of ECDSA signature validation in OpenSSL vs. libsecp256k1 over the last decade. Delving Bitcoin, 2025-11. <https://delvingbitcoin.org/t/comparing-the-performance-of-ecdsa-signature-validation-in-openssl-vs-libsecp256k1-over-the-last-decade/2087>
- [6] Wuille P. BIP-66: Strict DER signatures, 2015. <https://bips.dev/66/>
- [7] Pornin T. RFC 6979: Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). IETF, 2013.
- [8] Bernstein D J, Yang B Y. Fast constant-time gcd computation and modular inversion. IACR Transactions on CHES, 2019.
- [9] US Patent 7110538B2: Accelerated signature verification on an elliptic curve (GLV 自同态, 2020-09-25 到期) .